

```
.....mergeSort() called on: [7]
.....mergeSort() called on: [6]
The two halves merged into: [6, 7]
The two halves merged into: [3, 4, 6, 7]
The two halves merged into: [2, 3, 4, 5, 6, 7, 8, 9]
[2, 3, 4, 5, 6, 7, 8, 9]
```

As you can see from the output, the function divides the [2, 9, 8, 5, 3, 4, 7, 6] list into [2, 9, 8, 5] and [3, 4, 7, 6] and passes these to recursive `mergeSort()` calls. The first list is further split into [2, 9] and [8, 5]. That [2, 9] list is split into [2] and [9]. These single-value lists cannot be divided anymore, so we have reached our base case. These lists are merged back into sorted order as [2, 9]. The function divides the [8, 5] list into [8] and [5], reaches the base case, and then merges back into [5, 8].

The [2, 9] and [5, 8] lists are individually in sorted order. Remember, `mergeSort()` doesn't simply *concatenate* the lists into [2, 9, 5, 8], which would not be in sorted order. Rather, the function *merges* them into the sorted list [2, 5, 8, 9]. By the time the original `mergeSort()` call returns, the full list it returns is completely sorted.